Basic Docker commands

Course Code: INT332

1 Finding the version

One of the first things you want to know is how to find the installed docker version.

- $ docker --version

- Docker version 18.09.6, build 481bc77

2. Docker info

- Get detailed information about docker installed on the system including the kernel version, number of containers and images, etc.
- **$ docker info**

- Containers: 3
- Running: 1
- Paused: 0
- Stopped: 2
- Images: 3
- Server Version: 18.09.6
- Storage Driver: overlay2
- Backing Filesystem: extfs
- Supports d_type: true Native Overlay Diff: true Logging Driver:

3. Checking History

-
- Shows the history of a docker image with the image name mentioned in the command

$ docker history httpd

IMAGE	CREATED	CREATED BY	SIZE	COMMENT

4 . Downloading Image

When we need to pull the docker image from dockerhub (docker repository) then ,the following example of pulling the the image and here we are taking Apache HTTP server image.

$ docker pull httpd

```
Using default tag: latest

latest: Pulling from library/httpd

f5d23c7fed46: Pull complete

b083c5fd185b: Pull complete

bf5100a89e78: Pull complete

98f47fcaa52f: Pull complete

622a9dd8cfed: Pull complete

Digest: sha256:8bd76c050761610773b484e411612a31f299dbf7273763103edbda82acd73642

Status: Downloaded newer image for httpd:latest
```

5. **Images**

To list all the docker images pulled on the system with image details such as TAG/IMAGE ID/SIZE etc.

$ docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
httpd	latest	ee39f68eb241	2 days ago	154MB
hello-world	latest	fce289e99eb9	6 months ago	1.84kB
sequenceiq/hadoop-docker	2.7.0	789fa0a3b911	4 years ago	1.76GB

6.RUN

- The **‘docker run’** command is used to create and start a new container from a **Docker** image. The basic syntax of the **‘docker run’** command is as follows:
- **docker run [OPTIONS] IMAGE [COMMAND] [ARG…]**
- *where,*
- ***OPTIONS:*** *optional flags that configure the behaviour of the container, such as setting environment variables, mounting volumes, etc.*
- ***IMAGE:*** *the name of the Docker image to be run.*
- ***COMMAND:*** *the command to be executed when the container is started. If not provided, the default command specified in the Docker image will be used.*
- ***ARG:*** *optional arguments passed to the command.*

-d means

Options Available for executing Docker Run Command

- Some commonly used options for executing the docker run command inside a container:
- *-it: Runs the container in interactive mode, allowing you to interact with the container through the command line.*
- *-d: Runs the container in the background.*
- *-- name: Specifies a name for the container.*

-- rm: Automatically removes the container when it exits.

- *-p: Maps a host port to a container port.*
- -e / --env*: Sets an environment variable inside the container.*
- *-v: Mounts a host volume inside the container.*

1st Example : **Running a Simple Image:**

docker run httpd echo "Hello, World!"

- This command runs a new container from the **'*httpd* '** image and executes the **'echo'** command with the argument "Hello, World!".

2nd example

Running a Container with a Specific Name:

- ***$ docker run --name my-container httpd echo "Hello, World!"***

- ***--name : this option is to give the name of container***
- ***my-container: it is the name of container***
- ***Echo: it is the command***
- ***"hello,world!" – it is argument used***
- This command runs a new container with the specified name "my-container" from the **httpd** image and executes the echo command with the argument "Hello, World!".

What are Environment Variables

Environment variables are **key–value pairs** used by applications to:

- Read configuration
- Store secrets (usernames, passwords)
- Control application behavior **without changing the image**
- *Environment variables allow us to configure containers dynamically.*

Need of environment variables in Docker

Without -e:

- Configuration must be **hard-coded** in the image
- Need to rebuild image for every change

With -e:

- Same image → different behavior
- Secure & flexible configuration
- Widely used in **microservices & cloud deployments**

3rd Example : **Setting Environment:**

- **docker run -e MY_VAR=value httpd env**

- This command sets an environment variable **'MY_VAR'** to the value **"value"** and runs a new container from the **'httpd'** image.

The **'env'** command is executed to display the list of environment variables in the container.

2nd Example: Run a container from Image and display the variable name inside it.

docker run -it -e MY_NAME=Harpreet ubuntu bash

It will open the container and you can check the env variable inside it as:

Echo $ MY_NAME

Kindly note :

MY_NAME is created inside the container

Host system is not affected

Multiple Environment Variables

- docker run -e APP_ENV=production -e APP_VERSION=1.0 nginx

Same for MySQL Container

```
docker run -d \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=college \
  -e MYSQL_USER=admin \
  -e MYSQL_PASSWORD=admin123 \
  mysql:8
```

Without these variables, MySQL container will fail to start

Next is : Passing Environment Variable from Host System

Ex#1:

Step 1: Set variable on host

export APP_PORT=8080 (Ubuntu)

Powershell - $env:APP_PORT=8080

Check host variable - echo $env:APP_PORT

Step 2:

docker run -e APP_PORT=$env:APP_PORT nginx env

docker run -e APP_PORT=$APP_PORT nginx env

Another way : Using --env-file (Clean & Professional Way)

Create .env file

```
DB_HOST=localhost
DB_USER=root
DB_PASS=secret
```

Then Run container

```
docker run --env-file .env myapp
```

4th Example

$ docker run -it -d httpd

Here :

- **-it** flag enables interactive mode, so you can interact with the container's command line
- The -d flag runs the container in the background and the **httpd** argument specifies the image name to use for the container. This command will pull the **httpd** image from Docker Hub if it is not already present on your local machine, create a container based on that image, and start it in the background.

Passing Environment Variables to Containers

Host System

```
export API_KEY=12345
```

Environment Variable on Host

Sets Variable

Docker Engine

```
docker run -e API_KEY=12345 myapp
```

Passing Env Variable

Injects Variable

Docker Container

Inside Container:

$API_KEY=12345

Config Reads: API_KEY="12345"

Environment Variable in Container

Dynamic Configuration at Runtime

-p optional flag in run command

- **-p stands for *publish port*.**
  It maps a **port on your host machine** to a **port inside the container**.

In simple words:

- **-p allows users outside the container to access an application running inside the container.**

By default:

- Containers run in an **isolated network**
- Applications inside containers are **NOT accessible from the host/browser**
- So if a web server is running **inside a container**, you **must expose it** using -p.

Syntax of –p

docker run -p <HOST_PORT>:<CONTAINER_PORT> IMAGE

Part	Description
`HOST_PORT`	Port on your system (Laptop/Server)
`CONTAINER_PORT`	Port where app runs inside container

EX#1:

- Without –p

- docker run nginx

•Try opening browser: But it is Not accessible

As no port mapping done here

Now try to Run Nginx with -p

- docker run -p 8080:80 nginx

What happens internally?

Browser → localhost:8080 → Docker Host → Container:80 → Nginx

- docker run -d -p 3307:3306 --name mysql-test –e MYSQL_ROOT_PASSWORD=root123 mysql

docker exec -it mysql-test bash

mysql -h 127.0.0.1 -P 3306 -u root –p

7 What's running?

- To lists all the docker containers are running with container details.

  .

- **$ docker ps**

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
09ca6feb6efc	httpd	"httpd-foreground"	36 seconds ago	Up 33 seconds	80/tcp	suspicious_b

TASK #1:

- Run Ubuntu container
- Pass –e COLLEGE=CSE
- Show echo $COLLEGE
- Stop container then check what happened

8. ps -a

To list all the docker containers running/exited/stopped with container details

- **$ docker ps -a**

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
09ca6feb6efc	httpd	"httpd-foreground"	51 seconds ago	Up 49 seconds	80/tcp
2f6fb3381078	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -d"	2 weeks ago	Exited (137) 9 days ago	
9f397feb3a46	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -…"	2 weeks ago	Exited (255) 2 weeks ago	2122/tcp
9b6343d3b5a0	hello-world	"/hello"	2 weeks ago	Exited (0) 2 weeks ago	

9. **exec**

- Access the docker container and run commands inside the container in bash

$ docker exec -it 09ca6feb6efc bash/env

09ca6feb6efc --- it is Id of the container

```
root@09ca6feb6efc:/usr/local/apache2# ls

bin  build  cgi-bin  conf  error  htdocs  icons  include  logs                modules

root@09ca6feb6efc:/usr/local/apache2#
```

Type *exit* and press enter to come out of the container.

10. **Removing container**

- Remove the docker container with container id mentioned in the command.

$ docker rm 9b6343d3b5a0

Run the below command to check if the container got removed or not.

$ docker ps -a

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
09ca6feb6efc	httpd	"httpd-foreground"	About a minute ago	Up About a minute	80/tcp
2f6fb3381078	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -d"	2 weeks ago	Exited (137) 9 days ago	
9f397feb3a46	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -…"	2 weeks ago	Exited (255) 2 weeks ago	2122/tc

11. **Removing image**

- Remove the docker image with the docker image id mentioned in the command

  $ docker rmi fce289e99eb9

```
Untagged: hello-world:latest

Untagged: hello-world@sha256:41a65640635299bab090f783209c1e3a3f11934cf7756b09cb2f1e02147c6ed8

Deleted: sha256:fce289e99eb9bca977dae136fbe2a82b6b7d4c372474c9235adc1741675f587e

Deleted: sha256:af0b15c8625bb1938f1d7b17081031f649fd14e6b233688eea3c5483994a66a3

geekflare@geekflare:/home/geekflare$
```

..contd..

- To remove the image forcefully

$ docker rmi –f <image id>

docker rmi -f 92fa43a2ff60503fdf250

Untagged: httpd:latest

Untagged: httpd@sha256:7765977cf2063fec486b63ddea574faf8fbed285f2b17020fa7ef70a4926cdec

Deleted: sha256:92fa43a2ff60503fdf250ef0a16d4170aa2a47c4b5a60d427724283543a8792a

Practice Questions on Basic Docker Commands

- Run a Docker container named “DB-app" based on the “mongodb" image, and expose port 80 on the host to port 8082 on the container?

docker pull mongo

docker run -d --name DB-app -p 80:8082 mongo

- You are a DevOps engineer in a startup company. The development team asks you to quickly deploy a simple web page that displays the message:

Use the official httpd Docker image.

Run the container so it is accessible on port **8080** of the host machine.

Create or update an HTML file inside the container to display the required message.

Verify the output using a command-line tool.

Properly stop and remove the container after testing.

- docker pull httpd
- docker run -d --name my-apache -p 8080:80 httpd
- docker exec -it my-apache bash -c "echo '<h1>Hello from Docker Apache!</h1>' > /usr/local/apache2/htdocs/index.html"
- curl http://localhost:8080
- docker stop my-apache
- docker rm my-apache

3. How would you use the docker run command with -it, -e, -v, and --name to:

- Set an environment variable APP_ENV=production.

- Bind a local directory /app/data to /data inside the container.
- Name the container my_app.

- docker pull ubuntu
- mkdir C:\app\data
- docker run -it --name my_app -e APP_ENV=production -v C:\app\data:/data ubuntu /bin/bash
- echo $APP_ENV
- echo "Hello this is my first program" > /data/test.txt
- cat /data/test.txt
- echo "Second line added" >> /data/test.txt
- cat /data/test.txt
- exit

Docker container commands:

1. Container Lifecycle Management

- **Run a container** (creates and starts it)
- docker run -d -p 80:80 --name my_container nginx
- **Start a stopped container**
- docker start <container_id/name>
- **Stop a running container**
- docker stop <container_id/name>
- **Restart a container**
- docker restart <container_id/name>
- **Pause a container**
- docker pause <container_id/name>

…contd….

- **Unpause a paused container**
- docker unpause <container_id/name>
- **Kill a container (force stop)**
- docker kill <container_id/name>
- **Remove a container**
- docker rm <container_id/name>
- **Remove all stopped containers**
- docker container prune

2. Listing and Inspecting Containers

- **List all running containers**
- docker ps
- **List all containers (including stopped ones)**
- docker ps -a
- **Show detailed info about a container**
- docker inspect <container_id/name>
- **Display resource usage stats of a container**
- docker stats <container_id/name>
- **View container logs**
- docker logs <container_id/name>
- **Follow logs in real-time**
- docker logs -f <container_id/name>

3. Container Interaction with Host

- **Execute a command inside a running container**
- docker exec -it <container_id/name> <command>
- **Attach to a running container's interactive shell**
- docker attach <container_id/name>
- **Copy files between container and host**
  - From container to host:
  - docker cp <container_id>:<container_path> <host_path>
  - From host to container:
  - docker cp <host_path> <container_id>:<container_path>
- E.g:

Commands for CI with Host

- docker exec -it Container ID ls /
- docker exec -it Container ID /bin/bash
- mkdir /data
- echo "Hello Docker" > /data/test.txt
- exit
- docker attach Container ID
- docker cp Container ID:/data/test.txt C:\Users\HP\Desktop\
- docker cp C:\Users\HP\Desktop\test.txt Container ID :/data/sample.txt
- docker exec -it Container ID ls /data
- docker exec -it Container ID cat /data/sample.txt

Task

- Inside this same running container, you are required to:
- Create a new directory named /project.
- Inside /project, create a file called report.txt.
- Add the following text inside the file:

  This is our container interaction with our host machine.
- Verify the file content.
- Copy report.txt from the container to your Desktop.
- Create another file named notes.txt in the same /project directory.
- Append the text:
- Verify that both files exist inside /project.

- docker exec -it 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224 mkdir /project

- docker exec -it 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224 sh -c "echo 'Docker Lab Assignment' > /project/report.txt"
- docker exec -it 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224 sh -c "echo 'Container File Creation Test' >> /project/report.txt"
- docker exec -it 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224 cat /project/report.txt

- docker cp 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224:/project/report.txt C:\Users\HP\Desktop\

- docker exec -it 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224 sh -c "echo 'This is an additional file inside the same container.' > /project/notes.txt"
- docker exec -it 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224 ls /project
- docker exec -it 63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99187beaf3224 cat /project/notes.txt

Practice questions:

- 1. You need to start a new container using the nginx image while setting an environment variable ENV_MODE=production. Write the docker run command to achieve this.

- 2. You have a running container named my_app, but it's not behaving as expected. You want to check its logs to debug any errors. Write the command to view its logs and follow new log entries in real-time.
- 3. A container named web_server was stopped. Write the command to:a) Start the container again.b) Stop the container when needed.

Work on Volumes

Basic commands used to create,inpect,verify , delete/remove the volumes

1. docker volume create <volume name>
2. docker volume ls
3. docker volume inspect mydata
4. docker run -d -v <volumename>:<path where to store in image> --name <containername> <imagename>

- docker volume create mydata
- docker volume ls
- docker volume inspect mydata
- docker run -dit --name mycontainer -v mydata:/app/data ubuntu
- docker exec -it mycontainer ls /app/data
- docker inspect mycontainer
- docker rm -f mycontainer
- docker volume rm mydata
- docker volume prune

- Create a Docker volume named projectdata.
- Verify that the volume has been created successfully.
- Inspect the volume and identify:The mount point (where Docker stores it on the host system).
- Run an Ubuntu container in detached mode:

  Name the container project_container.
- Attach the volume projectdata inside the container at /app/data.
- Inside the running container:Create a file named report.txt inside /app/data.
- Add the text: This is my first volume.
- Stop and remove the container.
- Run a new Ubuntu container:Name it project_container_new.
- Attach the same volume projectdata at /app/data.
- Verify that report.txt still exists and contains the same data.

- docker volume create projectdata
- docker volume ls
- docker volume inspect projectdata
- docker run -dit --name project_container -v projectdata:/app/data ubuntu
- docker exec -it project_container sh -c "echo This is my first volume' > /app/data/report.txt“
- docker exec -it project_container cat /app/data/report.txt
- docker stop project_container
- docker rm project_container
- docker run -dit --name project_container_new -v projectdata:/app/data ubuntu
- docker exec -it project_container_new ls /app/data
- docker exec -it project_container_new cat /app/data/report.txt

Task 1

You are a DevOps engineer at a university. The web development team has built a simple HTML-based portal and wants it deployed using Docker.

- Use the official httpd image.
- Name the container college_portal.
- Run it in detached mode.
- Expose container port 80 to host port 8080.
- Set environment variable ENV=production.
- Attach a Docker volume named portaldata to store website files.
- Modify the index.html file inside the container.
- Verify the output in the browser.
- Check logs if the page is not loading.
- Stop and remove the container.
- Remove the volume and image after testing.

- docker pull httpd
- docker volume create portaldata
- docker run -dit --name college_portal -p 8080:80 -e ENV=production -v portaldata:/usr/local/apache2/htdocs httpd
- docker exec -it college_portal bash -c "echo '<h1>Welcome to University Portal - Production Mode</h1>' > /usr/local/apache2/htdocs/index.html“
- curl http://localhost:8080
- docker logs college_portal
- docker logs -f college_portal
- docker stop college_portal
- docker rm college_portal
- docker volume rm portaldata
- docker rmi httpd

Task 2

- Your backend team reports that their MySQL container is failing to start.
- Use mysql:8 image.
- Set required environment variables:
  - MYSQL_ROOT_PASSWORD
  - MYSQL_DATABASE
  - MYSQL_USER
  - MYSQL_PASSWORD
- Run in background.
- Expose port 3306 to host port 3307.
- Check logs to identify errors.
- Enter inside container to verify database.
- Stop and remove container after debugging.

- docker pull mysql:8
- docker run -d --name mysql_debug -p 3307:3306 -e MYSQL_ROOT_PASSWORD=root123 -e MYSQL_DATABASE=college -e MYSQL_USER=admin -e MYSQL_PASSWORD=admin123 mysql:8
- docker ps
- docker logs mysql_debug
- docker logs -f mysql_debug
- docker exec -it mysql_debug bash
- mysql -u root –p // # (Enter password: root123)
- exit
- Exit
- docker stop mysql_debug
- docker rm mysql_debug

Image creation in detail

- mkdir nginx-app
- cd nginx-app
- notepad index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Medium Level Docker App</title>
</head>
<body>
    <h1>Welcome to My Custom Docker Nginx App</h1>
    <p>This is a medium-level Docker project.</p>
</body>
</html>
```

- notepad default.conf

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
```

- notepad Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html
COPY default.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
```

- Dir

- docker build -t custom-nginx:v1 .
- docker run -d -p 8080:80 --name nginx-container custom-nginx:v1

- http://localhost:8080

- mkdir node-app
- cd node-app

- apt update
- apt install nano -y

- Create app.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Docker Node App Running!");
});

app.listen(3000, "0.0.0.0", () => {
  console.log("Server running on port 3000");
});
```

- Create package.json

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

- Create Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

Verify files

- Dir

app.js

package.json

Dockerfile- Rename-Item Dockerfile.txt Dockerfile

Build & run & verify

- docker build -t node-demo:v1 .

- Docker run -d -p 3000:3000 --name node-container node-demo:v1
- docker ps
- OPEN BROWSER

http://localhost:3000

Docker Node App Running!

Docker Networking

- Docker provides different network drivers to control how containers communicate.

A. Bridge Network (Default)

- Default network when you run a container.
- Containers on same bridge can communicate via IP.
- Best for standalone containers on single host.

```
# List networks
docker network ls

# Inspect bridge network
docker network inspect bridge

# Create custom bridge network
docker network create my_bridge_net

# Run container in custom bridge
docker run -d --name container1 --network my_bridge_net nginx
docker run -d --name container2 --network my_bridge_net alpine sleep 300

# Test communication
docker exec -it container2 ping container1
```

- Container shares host’s networking stack.
- No separate IP.
- Faster performance.
- Linux only.

```
docker run -d --network host nginx
curl http://localhost
```

- Used in Docker Swarm.
- Connects containers across multiple hosts.

```
# Initialize swarm
docker swarm init

# Create overlay network
docker network create -d overlay my_overlay

# Deploy service using overlay
docker service create --name web --network my_overlay -p 8080:80 nginx
```

- Docker has built-in DNS.
- Containers resolve each other by container name (if same custom network).
- Works only in user-defined bridge networks.

- docker network create app_net

- docker run -d --name backend --network app_net nginx
- docker run -it --name client --network app_net alpine sh

- ping backend

- Before custom networks, Docker used --link.
- Deprecated method. Use networks instead.

```
docker run -d --name db nginx
docker run -it --name app --link db:database alpine sh
```

```
# Create volume
docker volume create my_volume

# List volumes
docker volume ls

# Run container with volume
docker run -d \
  --name my_nginx \
  -v my_volume:/usr/share/nginx/html \
  nginx
```

- docker volume inspect my_volume
- docker volume rm my_volume

- Mounts host directory into container
- Useful for development

```
docker run -d \
  -v $(pwd)/html:/usr/share/nginx/html \
  -p 8080:80 \
  nginx
```

Volume Vs Bind Mount

Complete Example

- docker network create app_bridge
- docker network ls
- docker volume create mysql_data
- docker volume inspect mysql_data
- docker run -d --name mysql_db --network app_bridge -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=myapp -v mysql_data:/var/lib/mysql mysql:8
- docker run -d --name backend_app --network app_bridge -e DB_HOST=mysql_db node:18-alpine sh -c "apk add curl && sleep 300"

- docker exec -it backend_app ping mysql_db
- docker run -d --name frontend --network app_bridge -p 8080:80 nginx